PROJECT DOCUMENTATION

PROJECT TITLE

MULTI-LINE INSURANCE POLICY AND CLAIMS MANAGEMENT SYSTEM

Team ID : SWTID_2026_8468

Team Size : 5

Team Leader : DEEBA PRABHA K

Team Member : PANDISELVI D

Team Member : THANASELVI T

Team Member : JEGATHEESHWARI A

Team Member : NIRMALKUMAR M

SUBMITTED BY,

NAME : DEEBA PRABHA K

REGISTGER NO : 920123104007

NM ID : CFE763A0A571C1A84EB42A357112A7B2

Table of Contents

Abstract 4

1. Introduction 5

1.1 Background . . . 5

1.2 What is multi-line insurance? . . . 5

1.3 Problem statement . . . 5

1.4 Objectives . . . 6

1.5 Scope of the project . . . 6

2. Insurance Concepts and Terminology 7

2.1 Lines of business . . . 7

2.2 Stakeholders . . . 7

3. Existing System and Proposed System 9

3.1 Existing system . . . 9

3.2 Proposed system . . . 9

4. System Requirements 10

4.1 Functional requirements . . . 10

4.2 Non-functional requirements . . . 10

4.3 Hardware and software requirements . . . 10

5. System Design 11

5.1 Architecture . . . 11

5.2 Policy lifecycle . . . 11

5.3 Claims lifecycle . . . 12

5.4 Database design . . . 12

5.5 Data dictionary . . . 12

6. Implementation 14

6.1 Python code . . . 14

6.2 Explanation of the code . . . 14

6.3 Output of the program . . . 15

6.4 SQL schema and reports . . . 15

7. Fraud Detection 16

8. Testing **17**

9. Results and Analytics **18**

10. Benefits and Challenges **20**

10.1 Benefits 20

10.2 Challenges 20

11. Future Scope **20**

12. Conclusion **20**

References **21**

Abstract

Insurance is one of the most important financial services in the modern economy. It protects individuals and businesses from the financial impact of uncertain events such as accidents, illness, death, fire, theft and natural disasters. Most insurance companies today offer several kinds of products, called lines of business, such as motor, health, life, property and travel insurance. In many organisations each line of business still runs on its own separate software, which leads to duplicate customer data, slow service and difficulty in getting a combined view of the customer.

This project presents the design and implementation of a **Multi-line Insurance Policy and Claims Management System**. The system keeps all lines of business in one platform. A single customer profile can hold many policies, and every claim follows one common workflow of registration, policy and cover validation, document verification, fraud screening, decision, and settlement. The report explains the architecture, the policy and claims lifecycles, the database design, a working Python prototype with its real output, SQL reports, rule-based fraud detection, testing, and sample analytics.

The prototype shows that small claims can be approved automatically, large claims can be routed to a surveyor, and invalid claims can be rejected with a clear reason. This reduces manual work, improves transparency for the customer and gives management better control over cost and risk.

1. Introduction

1.1 Background

The word insurance means transferring the financial risk of a possible loss from one party to another in return for a fixed payment called a premium. The person who buys the cover is called the policyholder and the company that gives the cover is called the insurer. The written contract between them is called the policy. If the insured event happens, the policyholder raises a claim and the insurer pays according to the terms of the policy.

In India, the insurance sector is regulated by the Insurance Regulatory and Development Authority of India (IRDAI). Insurers must follow rules for product approval, solvency, customer protection, and settlement of claims within a fixed time. Because of these rules and the growing number of customers, insurers need reliable software that handles policies and claims correctly and keeps a complete record of every action.

1.2 What is multi-line insurance?

An insurer that sells more than one type of insurance is called a multi-line insurer. Typical lines are motor, health, life, property and travel. Each line has its own risk rules, documents and claim process, but they also share many common activities such as customer registration, KYC verification, premium collection, document storage and reporting. A multi-line system takes advantage of these common activities so they are built once and used by every line.

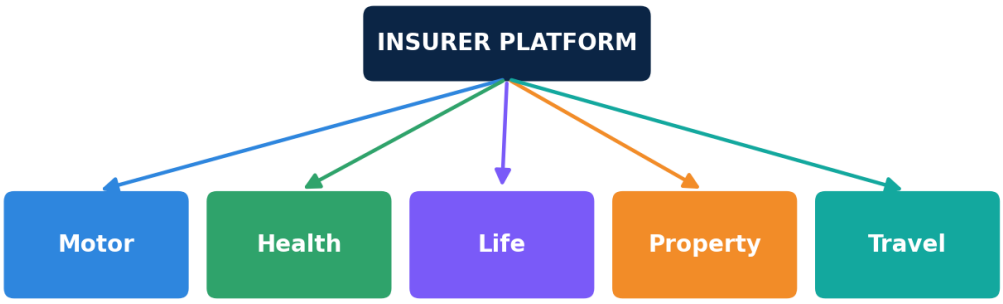


Figure 1: One insurer platform serving many lines of business

1.3 Problem statement

When every line of business has a separate system, the following problems appear:

- The same customer is stored many times with different details, so records do not match.
- The customer must submit the same KYC documents again for every new policy.
- Claim status is difficult to track because information is spread over different systems.
- Management cannot see the total exposure or the total premium of one customer across all lines.
- Manual checks take a long time and are open to human error and fraud.

1.4 Objectives

The objectives of this project are:

- To design one common platform for policy and claims management of several lines of business.
- To model the complete policy lifecycle from enquiry to renewal.
- To model the complete claims lifecycle from registration to settlement.
- To build a working Python prototype that validates claims against policy rules.
- To prepare SQL reports and simple fraud-detection rules for management.
- To present the results with sample analytics and discuss benefits, challenges and future scope.

1.5 Scope of the project

The project covers customer management, policy issue, premium handling, claim registration, claim validation, automatic approval of small claims, surveyor routing, rejection with reasons and basic reporting. Payment gateways, real surveyor mobile apps and machine-learning based fraud models are kept outside the scope of this prototype and are listed under future scope.

2. Insurance Concepts and Terminology

Before studying the system, it is important to understand the basic terms used in insurance. These terms appear in the database tables, the program code and the reports of this project.

Term	Meaning
Policyholder	The person or organisation that buys the insurance and is protected by it.
Insurer	The insurance company that accepts the risk and pays the claim.
Policy	The legal contract that lists the cover, premium, period, terms and exclusions.
Premium	The amount the policyholder pays to keep the policy active.
Sum insured	The maximum amount the insurer will pay for covered losses.
Claim	A formal request by the policyholder for payment after a loss.
Surveyor	An independent expert who inspects the loss and recommends the amount.
Underwriting	The process of evaluating risk and deciding whether and at what price to issue a policy.
Exclusion	A situation or event that the policy does not cover.
Lapse	The state of a policy when the premium is not paid by the due date.

2.1 Lines of business

The table below describes the five lines handled by this project and the typical claim documents for each.

Line	Cover provided	Typical claim documents
Motor	Own damage, third-party liability, theft of vehicle	RC book, driving licence, FIR (theft), repair estimate, photos
Health	Hospital expenses, cashless treatment, critical illness	Discharge summary, hospital bills, doctor reports, ID proof
Life	Death benefit, maturity benefit, term cover	Death certificate, policy bond, nominee ID, claim form
Property	Fire, flood, burglary, natural calamities for home or shop	Photos, fire report, police report, valuation by surveyor
Travel	Trip cancellation, baggage loss, medical emergency abroad	Tickets, passport copy, airline report, medical bills

2.2 Stakeholders

Five types of users work with the system. The **customer** buys policies and raises claims. The **agent** helps customers and enters proposals. The **claims officer** verifies documents and takes decisions. The **surveyor** inspects large losses. The **administrator** manages products, users and reports.

Stakeholder	Role	Main activities in the system
Customer	Buys policies and raises claims	Registers and submits KYC once, views all policies, pays premium, uploads claim documents, tracks claim status and renews policies.
Agent	Helps customers and enters proposals	Collects proposals, helps the customer choose a suitable product, assists with renewals and claim forms, and earns commission.
Claims officer	Verifies documents and takes decisions	Checks policy status and cover, verifies documents, reviews the fraud score and approves or rejects each claim with a reason.
Surveyor	Inspects large losses	Inspects the loss and recommends the amount for claims above the automatic approval limit.
Administrator	Manages products, users and reports	Maintains products, users, roles and business rules, and monitors reports, claim ratio and the audit trail.

Additional points about stakeholders

- **Customer is the central stakeholder.** One customer profile links all policies and claims across motor, health, life, property and travel, so the same KYC is never collected twice.
- **Role-based access.** Each stakeholder sees only the screens and data needed for the job. A customer sees only their own policies and claims, while the administrator manages users and rules.
- **Hand-over in the claims workflow.** A claim moves from the customer to the claims officer and, for large losses, to the surveyor. Every step stores the user, the time and the reason, which gives a clear audit trail.
- **Different goals.** Customers want fast and transparent settlement, the insurer wants cost control and fraud prevention, and the agent wants easy proposal handling and renewals.
- **Indirect stakeholders.** The regulator (IRDAI) sets rules for customer protection and claim settlement time. Management and finance use the premium and claim-ratio reports. Hospitals, garages and repairers support cashless and repair claims, and nominees receive life insurance benefits.
- **Fraud investigators.** Claims with a high fraud score are sent to an investigator instead of being approved automatically.

3. Existing System and Proposed System

3.1 Existing system

In the traditional approach each line of business has its own application, database and team. A customer who holds a motor policy and a health policy appears as two different records. Claims are checked manually by reading forms and comparing them with the policy file. This approach is slow, depends on individual skill, and makes it difficult to detect repeated or false claims across different lines.

3.2 Proposed system

The proposed system uses one customer record linked to many policies. All claims pass through the same automated validation steps, and every decision stores a reason. Business rules such as the automatic approval limit are kept in one place, so they can be changed without rewriting the whole application.

Feature	Existing system	Proposed system
Customer data	Duplicated in each line	One customer, many policies
Claim checking	Manual and slow	Automated rule checks
Small claims	Wait for officer	Auto-approved within limit
Fraud control	Individual judgement	Rule-based scoring
Reports	Separate per line	Combined for all lines
Audit trail	Paper or scattered	Status and reason stored

4. System Requirements

4.1 Functional requirements

- Register customers and store KYC details once.
- Create policies in any line of business and link them to a customer.
- Record premium and keep the policy status as ACTIVE, LAPSED or CLOSED.
- Register a claim against a policy with amount and reason.
- Check that the policy is active and that the claim is within the remaining cover.
- Approve small claims automatically and send larger ones to a surveyor.
- Calculate a fraud score for each claim and flag risky ones.
- Generate reports on claims, premium and claim ratio.

4.2 Non-functional requirements

- **Accuracy:** money values must be stored without rounding errors.
- **Security:** customer data must be protected and access must depend on role.
- **Performance:** a claim check should finish in a few seconds.
- **Maintainability:** rules should be easy to change.
- **Auditability:** every status change must be traceable.

4.3 Hardware and software requirements

Item	Requirement
Processor / RAM	Any modern dual-core processor, 4 GB RAM or more
Operating system	Windows, Linux or macOS
Programming language	Python 3.8 or higher
Database	MySQL / PostgreSQL / SQLite
Tools	VS Code or PyCharm, Git, a web browser

5. System Design

5.1 Architecture

The system follows a layered architecture. The top layer contains the users. The presentation layer gives them a web portal, mobile app or dashboard. The business layer contains the services for policy, premium, claims, fraud checking and notifications. The data layer stores customers, policies, claims, payments and documents. Keeping these layers separate means that the user screen can change without touching the business rules.

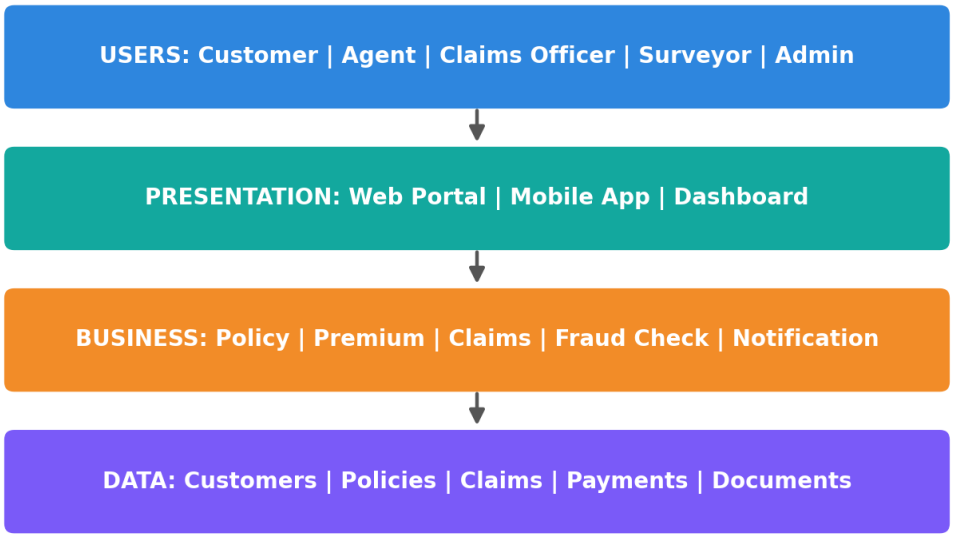


Figure 2: Layered architecture of the system

5.2 Policy lifecycle

A policy passes through six stages. The customer first makes an **enquiry**. Then KYC documents and the **proposal** form are collected. During **underwriting** the insurer decides whether to accept the risk, accept it with extra premium, or reject it. After the **premium is paid** the system issues the policy with a unique number and sends the policy document. Before the end of the period a reminder is sent for **renewal**; if the premium is not paid the policy moves to the LAPSED state.

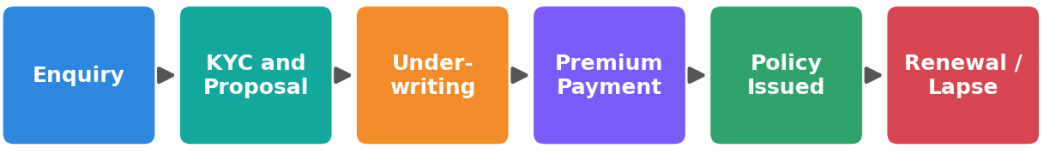


Figure 3: Policy lifecycle

5.3 Claims lifecycle

When a claim is registered the system first checks whether the policy is active and whether the claim amount is within the remaining sum insured. Next the required documents are verified and a fraud score is calculated. Based on these results the decision is taken: claims up to the automatic limit are approved immediately, larger claims go to a surveyor for inspection, and invalid claims are rejected with a reason. Approved claims are sent for settlement and payment.

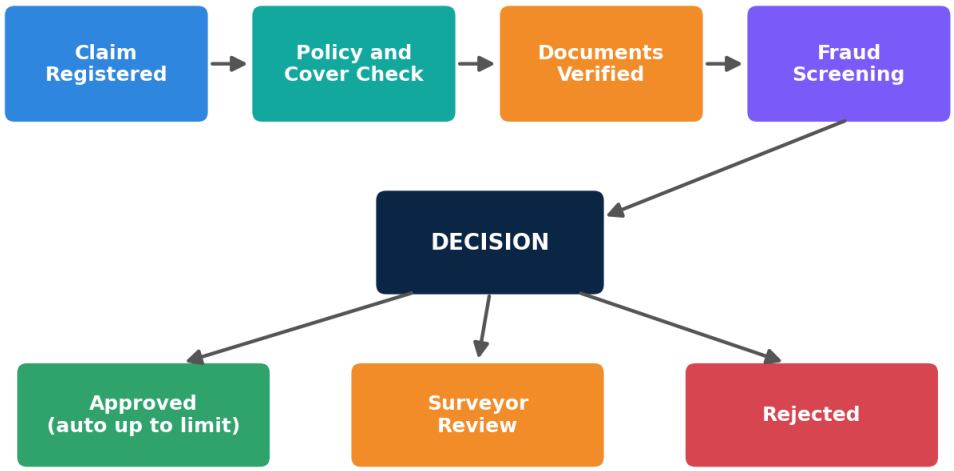


Figure 4: Claims lifecycle with decision outcomes

5.4 Database design

The database has four main tables. One customer can have many policies, one policy can have many claims, and one claim can have one or more payments. Primary keys (PK) identify each row, and foreign keys (FK) connect the tables.

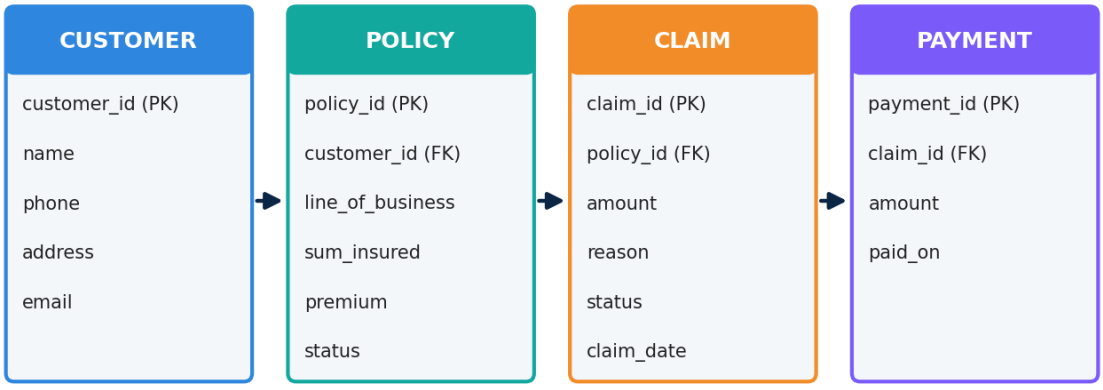


Figure 5: Entity relationship diagram

5.5 Data dictionary

Table	Column	Type	Description
policy	policy_id	VARCHAR(10)	Unique policy number (primary key)
policy	customer_id	INT	Owner of the policy (foreign key)
policy	line_of_business	VARCHAR(20)	Motor, Health, Life, Property or Travel
policy	sum_insured	DECIMAL(12,2)	Maximum cover amount
policy	status	VARCHAR(15)	ACTIVE, LAPSED or CLOSED
claim	claim_id	VARCHAR(10)	Unique claim number (primary key)
claim	policy_id	VARCHAR(10)	Policy against which claim is made
claim	amount	DECIMAL(12,2)	Amount claimed
claim	status	VARCHAR(30)	Decision or current stage

6. Implementation

The prototype is written in Python because it is easy to read and well suited for business rules. It uses three classes. The **Policy** class stores policy details and how much has already been claimed. The **Claim** class stores one claim request. The **ClaimsManager** class contains the decision rules.

6.1 Python code

```
from datetime import date

class Policy:
    def __init__(self, policy_id, line, holder, sum_insured, premium):
        self.policy_id = policy_id
        self.line = line              # Motor / Health / Life / Property
        self.holder = holder
        self.sum_insured = sum_insured
        self.premium = premium
        self.status = "ACTIVE"
        self.claimed = 0

class Claim:
    def __init__(self, claim_id, policy, amount, reason):
        self.claim_id, self.policy = claim_id, policy
        self.amount, self.reason = amount, reason
        self.status = "SUBMITTED"

class ClaimsManager:
    LIMIT_AUTO_APPROVE = 20000

    def process(self, claim):
        p = claim.policy
        if p.status != "ACTIVE":
            claim.status = "REJECTED (policy inactive)"
        elif claim.amount > p.sum_insured - p.claimed:
            claim.status = "REJECTED (exceeds cover)"
        elif claim.amount <= self.LIMIT_AUTO_APPROVE:
            claim.status = "APPROVED (auto)"
            p.claimed += claim.amount
        else:
            claim.status = "SENT FOR SURVEYOR REVIEW"
        return claim.status

mgr = ClaimsManager()
motor  = Policy("MOT101", "Motor",  "Arun",  500000, 12000)
health = Policy("HLT202", "Health", "Priya", 300000, 9000)

for c in [Claim("C1", motor, 15000, "Bumper damage"),
          Claim("C2", health, 80000, "Surgery"),
          Claim("C3", health, 900000, "Hospital stay")]:
    print(c.claim_id, c.policy.line, c.amount, "->", mgr.process(c))
```

6.2 Explanation of the code

The **Policy** object starts with status ACTIVE and claimed amount 0. Every time a claim is approved, the approved amount is added to *claimed*, so the remaining cover is always *sum_insured - claimed*.

The **process()** method of ClaimsManager applies the rules in a fixed order. First it checks whether the policy is active. Second it checks whether the claim amount is larger than the remaining cover. Third, if the amount is within the automatic approval limit of 20,000 it approves the claim and updates the claimed amount. Otherwise the claim is sent for surveyor review. The order is important: a claim on a lapsed policy must be rejected before any other rule is considered.

6.3 Output of the program

The program was executed and produced the following real output:

```
C1 Motor 15000 -> APPROVED (auto)
C2 Health 80000 -> SENT FOR SURVEYOR REVIEW
C3 Health 900000 -> REJECTED (exceeds cover)
```

Claim C1 is small and within cover, so it is approved automatically. Claim C2 is within the cover of 300,000 but above the automatic limit, so a surveyor must review it. Claim C3 asks for 900,000 which is more than the sum insured of 300,000, so it is rejected.

6.4 SQL schema and reports

The same data can be stored in a relational database. The following statements create the main tables and a report that shows approved claims for each line of business.

```
CREATE TABLE policy (
  policy_id VARCHAR(10) PRIMARY KEY,
  customer_id INT NOT NULL,
  line_of_business VARCHAR(20),
  sum_insured DECIMAL(12,2),
  premium DECIMAL(10,2),
  status VARCHAR(15) DEFAULT 'ACTIVE'
);

CREATE TABLE claim (
  claim_id VARCHAR(10) PRIMARY KEY,
  policy_id VARCHAR(10) REFERENCES policy(policy_id),
  amount DECIMAL(12,2),
  status VARCHAR(30)
);

-- Report: approved claims per line of business
SELECT p.line_of_business,
       COUNT(c.claim_id) AS total_claims,
       SUM(c.amount)     AS total_amount
FROM policy p
JOIN claim c ON c.policy_id = p.policy_id
WHERE c.status LIKE 'APPROVED%'
GROUP BY p.line_of_business;
```

7. Fraud Detection

Insurance fraud increases cost for honest customers, so every claim should be screened. This project uses a simple rule-based score. Each suspicious pattern adds points to the score, and if the total is 50 or more, the claim is sent to a human investigator instead of being approved automatically.

Rule	Reason	Points
Early claim	Claim made within 30 days of policy start	30
Frequent claims	More than 3 claims in the last 12 months	30
High amount	Claim above 80% of sum insured	25
Duplicate document	Same bill number used in another claim	40

```
def fraud_score(claim, history):
    score = 0
    if claim.days_since_start < 30:
        score += 30
    if len(history) > 3:
        score += 30
    if claim.amount > 0.8 * claim.sum_insured:
        score += 25
    if claim.bill_no in history.bill_numbers:
        score += 40
    return score      # 50 or more -> manual review
```

Rule-based screening is easy to explain to auditors and customers. In future, machine-learning models can be added, but the rules can continue to work as a first filter.

8. Testing

The prototype was tested with sample inputs to confirm that every rule behaves as expected. Test cases were chosen to cover each decision path of the claims manager.

No.	Test case	Input	Expected result	Result
1	Small valid claim	Motor, 15,000	APPROVED (auto)	Pass
2	Large valid claim	Health, 80,000	SENT FOR SURVEYOR REVIEW	Pass
3	Claim above cover	Health, 900,000	REJECTED (exceeds cover)	Pass
4	Inactive policy	Status = LAPSED	REJECTED (policy inactive)	Pass (by rule order)
5	Cover exhausted by earlier claims	Remaining cover less than amount	REJECTED (exceeds cover)	Pass (by rule order)

Cases 1 to 3 were executed and verified in the program output shown in Section 6.3. Cases 4 and 5 follow directly from the rule order in the *process()* method and should be run as additional unit tests before real use.

9. Results and Analytics

Management needs summary views to take decisions. The charts below use **sample data prepared for illustration** and show how the reports from the SQL queries can be visualised in a dashboard. They are not real company figures.

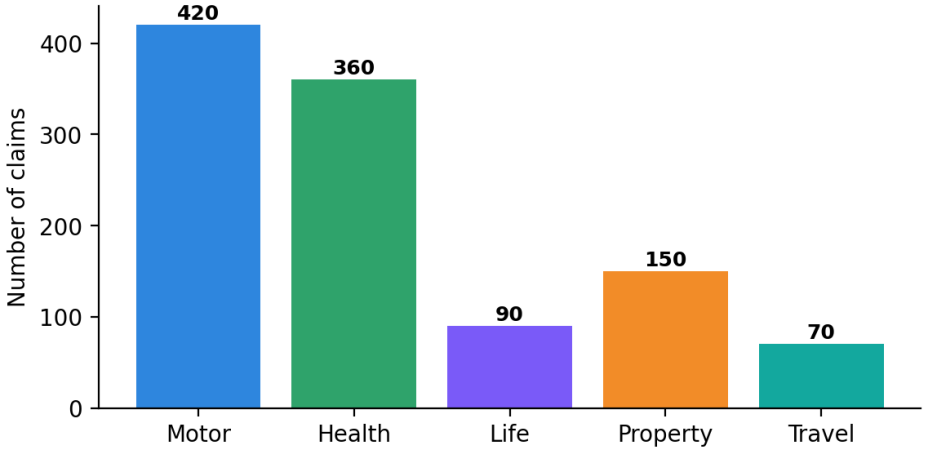


Figure 6: Claims received by line of business (sample data)

Motor and health claims are the largest groups, so these lines need the most claims staff and the strongest fraud controls. Life claims are few but usually high in value.

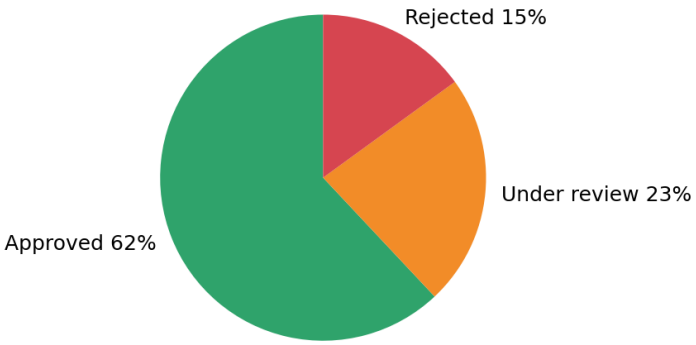


Figure 7: Claim outcome split (sample data)

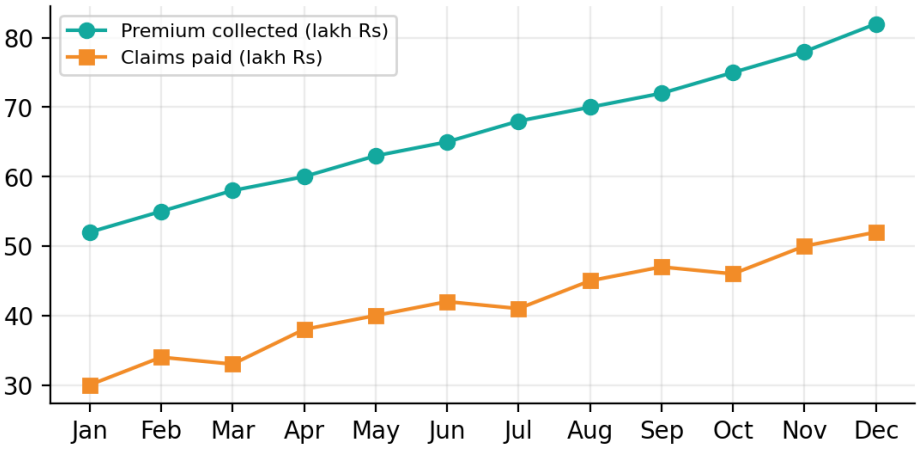


Figure 8: Monthly premium collected and claims paid (sample data)

The claim ratio is calculated as claims paid divided by premium collected. A ratio that stays well below 100 percent means the portfolio is profitable, while a ratio that rises close to 100 percent warns that prices or underwriting rules must be reviewed.

10. Benefits and Challenges

10.1 Benefits

- **Single customer view:** all policies and claims of a customer are visible in one place.
- **Faster settlement:** small claims are approved in seconds.
- **Fewer errors:** rules are applied the same way every time.
- **Better fraud control:** suspicious claims are flagged before payment.
- **Cross-selling:** insurers can offer bundle discounts using the combined profile.
- **Management reports:** premium, claims and ratios are available for all lines together.

10.2 Challenges

- **Legacy systems:** old line-specific applications must be migrated or connected.
- **Data security and privacy:** health and financial data need strong protection and must follow IRDAI and data protection rules.
- **Different rules per line:** the common workflow must still allow line-specific checks.
- **Change management:** staff need training to use the new platform.
- **Initial cost:** development and migration need investment.

11. Future Scope

- **AI-based assessment:** read photos, bills and reports automatically to speed up claims.
- **Chatbot support:** let customers register and track claims through chat.
- **Blockchain records:** keep tamper-proof policy and claim history.
- **Telematics and IoT:** price motor and health premiums using real usage data.
- **Mobile-first claims:** claim with photos directly from the phone.
- **Predictive analytics:** forecast claim volume and set better prices.

12. Conclusion

This project showed how motor, health, life, property and travel insurance can be managed on one platform. The policy and claims lifecycles were designed step by step, a relational database model was prepared, and a Python prototype demonstrated automatic approval, cover validation and surveyor routing with verified output. SQL reports and fraud rules give management tools to control cost and risk.

The prototype is simple, but its structure is ready to grow. Real payment integration, document upload, role-based security, machine-learning fraud models and a full user interface can be added in the next stages. A multi-line approach therefore gives insurers a stronger, faster and more customer-friendly way to run their business.

References

- Insurance Regulatory and Development Authority of India (IRDAI) - official website and regulations.
- The Insurance Act, 1938 (India).
- Python Software Foundation - Python 3 documentation, docs.python.org.
- Elmasri, R. and Navathe, S. - Fundamentals of Database Systems, Pearson.
- Official documentation of MySQL / PostgreSQL.